Hallucination to Consensus: Multi-Agent LLMs for End-to-End JUnit Test Generation

QINGHUA XU, Research Ireland Lero Centre for Software and University of Limerick, Ireland
GUANCHENG WANG, Research Ireland Lero Centre for Software and University of Limerick, Ireland
LIONEL BRIAND, Research Ireland Lero Centre for Software and University of Limerick, Ireland and University of Ottawa, Canada
KUI LIU, Software Engineering Application Technology Lab, Huawei, China

Unit testing plays a critical role in ensuring software correctness. However, writing unit tests manually is labor-intensive, especially for strongly typed languages like Java, motivating the need for automated approaches. Traditional methods primarily rely on search-based or randomized algorithms to generate tests that achieve high code coverage and produce regression oracles, which are assertions derived from the program's current behavior rather than its intended functionality. Recent advances in large language models (LLMs) have enabled oracle generation from natural language descriptions, aligning better with user requirements. However, existing LLM-based methods often require LLM fine-tuning or rely on external tools such as EvoSuite for test prefix generation, making them costly or cumbersome to apply in practice.

In this work, we propose CANDOR, a novel end-to-end, prompt engineering-based LLM framework for automated unit test generation in Java. CANDOR orchestrates multiple specialized LLM agents to collaboratively generate complete JUnit tests, including both high-quality test prefixes and accurate oracles. To mitigate the notorious hallucinations in LLMs and improve oracle correctness, we introduce a novel strategy that engages multiple reasoning LLMs in a panel discussion and generates accurate oracles based on consensus. Additionally, to reduce the verbosity of reasoning LLMs' outputs, we propose a novel dual-LLM pipeline to produce concise and structured oracle evaluations.

Our experiments on the HumanEvalJava and more complex LeetCodeJava datasets show that CANDOR is comparable with EvoSuite in generating tests with high code coverage and clearly superior in terms of mutation score. Moreover, our prompt engineering-based approach CANDOR significantly outperforms the state-of-the-art, fine-tuning-based oracle generator TOGLL by at least 21.1 percentage points in oracle correctness on both correct and faulty source code. Further ablation studies confirm the critical contributions of key agents in improving test prefix quality and oracle accuracy.

CCS Concepts: • **Software and its engineering → Software testing and debugging**; • **Computing methodologies → Natural language generation**; **Causal reasoning and diagnostics**.

Additional Key Words and Phrases: Unit test, Large Language Model, Test Oracle Generation

1 Introduction

Unit testing is a software testing activity wherein individual units of code are tested in isolation [68]. Unit testing plays a crucial role in modern software development because it helps software developers identify and fix defects in early phases [10]. However, manually creating unit tests is laborious and necessitates substantial domain

Authors' Contact Information: Qinghua Xu, qinghua.xu@ul.ie, Research Ireland Lero Centre for Software and University of Limerick, Limerick, Ireland; Guancheng Wang, guancheng.wang@ul.ie, Research Ireland Lero Centre for Software and University of Limerick, Limerick, Ireland; Lionel Briand, lionel.briand@ul.ie, Research Ireland Lero Centre for Software and University of Limerick, Limerick, Ireland and University of Ottawa, Ottawa, Canada; Kui Liu, brucekuiliu@gmail.com, Software Engineering Application Technology Lab, Huawei, Hangzhou, China.

ACM 1557-7392/2026/3-ART
https://doi.org/10.1145/3803418

expertise to craft high-quality tests [16, 17]. Consequently, developers often forgo writing tests for their code altogether. As reported in [31, 68], only 17 % of 82,447 studied Github projects contained test files.

To reduce the burden of manual test writing, various solutions have been proposed to automatically generate test suites [28, 54, 63, 80, 82]. A test suite typically comprises a set of test cases, where each test case is composed of a *test prefix* — inputs to the system under test (SUT) — and a *test oracle* that verifies whether the actual behavior matches the expected behavior of the SUT [39].

Prior works in automated test generation predominantly focus on **test prefix generation**, with a goal of maximizing code coverage [62, 77]. Traditional techniques in this area include fuzzing [26, 49], feedback-directed random test generation [7, 15, 52, 53, 60], dynamic symbolic execution [12, 30, 61, 75], and search/evolutionary algorithm-based approaches [27, 28]. Such methods are generally designed to maximize code coverage but often struggle to produce meaningful and human-readable test prefixes, primarily due to their limited understanding of the semantics of focal methods [87]. Therefore, recent research has shifted to leveraging large language models (LLMs) to produce more practical and meaningful test prefixes [22]. However, Tang et al. [72] and Yang et al. [84] found that search-based approaches like EvoSuite [28] remain the best test prefix generator, substantially outperforming LLM-based approaches by approximately 20 % in terms of code coverage on benchmark datasets.

In contrast to the increasing attention on prefix generation, **test oracle generation** remains underexplored. Most existing works circumvent this challenge by generating *regression oracles* [27, 28, 43, 47, 57, 59], which are derived from the system's current behavior. *Regression oracles* assume the existing code implementation is correct and simply record its behavior as the expected outcome. As such, they are primarily useful for detecting behavioral changes across software versions, not for validating functional correctness against intended specifications.

A more rigorous and practical alternative to *regression oracles* is *specification-based oracles*, which are derived from specifications. However, generating correct *specification-based oracles* poses significant challenges, primarily because it necessitates rigorous reasoning about program semantics and expected behaviors. To that end, several works have proposed leveraging LLMs to generate oracles from specifications. Empirical studies have shown that LLMs can achieve state-of-the-art (SOTA) performance in oracle generation for weakly typed languages like JavaScript [59] and Python [79]. However, generating test oracles for strongly typed languages such as Java remains significantly more challenging. This is due to the need for strict adherence to type constraints, language-specific syntax, and more complex execution semantics. Several works have attempted to address these challenges, using either fine-tuning (FT) or prompt engineering (PE) strategies [24, 35, 39, 56, 68]. FT adapts LLMs to specific datasets while PE leverages off-the-shelf LLMs through carefully crafted prompts. FT-based approaches generally yield higher oracle correctness compared to prompt-engineering-based approaches. For instance, LLM-Empirical [68], as a representative state-of-the-art PE approach, demonstrates limited oracle correctness compared to FT-based approaches. In particular, TOGLL achieves the best oracle correctness by fine-tuning CodeParrot LLM on the SF110 dataset [39]. However, we identify three key limitations of TOGLL: (1) It relies on EvoSuite to produce initial test scaffolds, which inherits the known limitations of EvoSuite in generating human-readable test cases and reasoning about complex programs [47]; Fraser and Arcuri [29] reported side effects (e.g., "state pollution") on 50 % of tests generated by EvoSuite [5]. Moreover, EvoSuite has not been maintained since 2021, resulting in incompatibility issues with newly-developed projects [14]. (2) It requires fine-tuning on version-specific Java data, which is costly to collect, often unavailable, and may not generalize well to new projects. Specifically, TOGLL is fine-tuned on test cases generated using EvoSuite v1.0.6, Java 8, and Ant Builder. Applying TOGLL to other environments (e.g., EvoSuite 1.2.0, Java 11, Maven Builder) would necessitate collecting new datasets and re-fine-tuning or domain adaptation techniques; otherwise, its performance would likely degrade substantially [65]. (3) The optimal configuration of TOGLL takes both Java docstrings and method signatures as input. Docstrings are informal natural language specifications that are ubiquitous in modern software [24], whereas method signatures are less common in most software code bases. As demonstrated in [39], however, the absence of method signatures leads to a 20 % decrease in oracle correctness. Thus, reliance on method

signatures makes TOGLL less robust when such information is incomplete or unavailable, which represents a practical limitation of the approach.

To address the afore-mentioned challenges in test prefix and oracle generation, we propose a novel approach, CANDOR, for automated JUnit test generation using off-the-shelf LLMs. We highlight that CANDOR is the first multi-agent-based framework for testing Java methods using only off-the-shelf LLMs, thereby laying the foundation for future research on Java test generation. In particular, CANDOR aims to generate test prefixes with at least comparable code coverage and mutation scores as EvoSuite. More importantly, it aims to derive accurate specification-based oracles directly from informal Java docstrings (hereafter referred to as *descriptions*), thereby compensating for the lack of formal specifications in most software systems.

For test prefix generation, CANDOR coordinates multiple specialized LLM agents—*Initializer*, *Planner*, *Tester*, and *Inspector*—to construct and refine test files iteratively. These agents are responsible for generating an initial test scaffold, designing test plans to improve coverage, producing executable test cases, and inspecting the quality of those test cases. At this stage, oracles are generated directly from the source code, which may be faulty in practice, and thus the resulting assertions—referred to as tentative oracles—may be incorrect.

To ensure oracle correctness, CANDOR introduces a novel ensemble-based strategy inspired by David Hume's quote, "*Truth springs from arguments amongst friends*", which underscores that constructive dialogue and diverse perspectives lead to robust conclusions. In this spirit, CANDOR employs a panel discussion-style approach in which multiple *Panelist* agents, powered by reasoning LLMs such as DeepSeek R1 [33], independently evaluate tentative oracles against requirements derived from the natural-language description of the SUT. Noticeably, CANDOR requires only an informal description of the SUT's function instead of a formal specification, following the practices in prior works [35, 39, 72]. This can be attributed to recent findings suggesting that LLMs are robust to noisy natural-language instructions. However, effective application of LLMs is non-trivial. Despite their effectiveness, LLMs often suffer from verbose and redundant outputs, known as the "overthinking phenomenon" [71]. To address this, CANDOR introduces a dual-LLM pipeline, where a basic LLM (i.e., without reasoning capability) agent *Interpreter* extracts and formats key insights from each *Panelist*'s output. Finally, a *Curator* agent, also a basic LLM, aggregates these interpretations from the pipelines, determines necessary corrections, and generates accurate oracles through consensus. This design mitigates hallucination and uncertainty in test oracles by enabling multiple LLM agents to cross-validate tentative oracles and reach a consensus, ensuring that the final oracles align more closely with the intended behavior described in the specification.

We evaluate CANDOR in terms of the quality of both test prefixes and oracles on two benchmarking datasets: HumanEvalJava [8] and LeetCodeJava. HumanEvalJava is an extensively studied dataset in the domain of automated test/code generation [24, 35, 44, 46, 48, 68, 74, 86]. To further assess CANDOR's generalizability, we introduce a new dataset, LeetCodeJava, we constructed from the popular online judgement system LeetCode [2], which consists of solutions to programming problems with varying difficulty levels. Experimental results show that CANDOR is comparable with EvoSuite in generating high-coverage test prefixes and clearly better in terms of mutation score, while producing accurate specification-based oracles. CANDOR also outperforms the SOTA, fine-tuning-based approach TOGLL by at least 21.1 percentage points in terms of oracle correctness. We remark that this comparison is inherently unfavorable to CANDOR as it uses off-the-shelf LLMs, while TOGLL is fine-tuned with extra data. Fine-tuning adapts LLMs with domain-specific data, generally exhibiting higher effectiveness than prompt engineering in various tasks [40, 55, 66, 67]. Nevertheless, because TOGLL is the state-of-the-art approach for Java oracle generation, we include this comparison for completeness. We also assess the individual contribution of the key agents in CANDOR. Experimental results show that removing *Planner* leads to a substantial decrease in line coverage (0.050), branch coverage (0.046), and mutation score (0.070), respectively. Removing the *Requirement Engineer* agent and the panel discussion decreases oracle correctness by at least 0.007 and 0.067, respectively.

In summary, our contributions are as follows.

(1) To the best of our knowledge, CANDOR is the first multi-agent LLM framework for end-to-end JUnit test generation, where specialized agents collaborate to generate, validate, and refine test cases iteratively. Unlike most existing approaches, CANDOR does not rely on expensive LLM fine-tuning or external tools like EvoSuite, which has not been maintained since 2021 and is difficult to adapt to new language features.
(2) We introduce a novel panel discussion-inspired strategy for test oracle generation, mitigating LLMs' hallucination and uncertainty.
(3) To address the "overthinking phenomenon" of reasoning LLMs, we design a novel dual-LLM pipeline to extract concise oracle evaluations from their verbose output.
(4) Experimental results on two benchmarks, HumanEvalJava and LeetCode, demonstrate that CANDOR compares favorably with EvoSuite in generating high-quality test prefixes with comparable line/branch coverage and clearly higher mutation scores, while producing specification-based oracles with high accuracy, substantially outperforming a SOTA baseline by at least 21.1 percentage points.

The rest of this paper is organized as follows. Section 2 introduces a running example that facilitates the presentation of CANDOR in Section 3. Section 4 describes the experimental design, and Section 5 reports the corresponding results. Section 6 discusses key insights and threats to validity. Section 7 reviews related work, and Section 8 concludes the paper.

2 Running Example

In this section, we present the running example in Figure 1, which is adapted from the HumanEvalJava dataset [13]. For each subject under test (SUT), we assume the availability of two relevant pieces of information: the *source code* and its natural language *description*.

Description. This part provides a natural language summary of the SUT's main functionality. We assume such a description is available, as programmers typically provide such documentation explaining their code [68]. This description should include the input, the core implementation logic, and the expected output. As depicted in Figure 1, the *description* specifies that the SUT should take a list of integers as input, implement element-wise transformation, and return the sum of the transformed list as output. For the ith element $lst[i]$, the transformation rule is as in Equation 1.

$$lst[i] = \begin{cases} lst[i]^2, & \text{if } (i\%3) == 0 \\ lst[i]^3, & \text{if } (i\%3)! = 0 \text{ and } (i\%4) == 0 \\ lst[i], & \text{otherwise} \end{cases} \tag{1}$$

Source Code. The *source code* is an implementation of the logic described in the *description*, using the specified input and producing the expected output. For example, as shown in the second box of Figure 1, it defines a class "SumSquares1" with a static method "sumSquares". In this example, the implementation is faulty: the condition "$i\%3! = 0$" from Equation 1 is incorrectly written as "$i\%3 == 0$".

This example represents a realistic and challenging test generation scenario, where the SUT is faulty and thus unreliable for deriving an oracle. In real-world development, especially during early-stage or iterative coding, it is common for source code to contain latent bugs or incomplete logic [25]. The goal of this paper is to generate high-quality tests with accurate oracles by combining both the *source code* and its natural language *description*, thus ensuring that the generated oracles are aligned with the intended functionality.

3 Methodology

This section demonstrates the workflow of CANDOR, comprising three steps: *Initialization*, *Test Prefix Generation*, and *Oracle Fixing*. In the *Initialization* step, an initial test file v_0 is generated from the source code and iteratively refined by fixing syntactic errors found by a validation process, resulting in a syntactically correct test file v_1. This file, typically containing only a small number of test cases (often ≤ 3), serves as a template for the subsequent

```
/**
*
* This function will take a list of integers.
* For all entries in the list, the function shall square the integer entry
if its index is a multiple of 3 and will cube the integer entry if its
index is a multiple of 4 and not a multiple of 3.
* The function will not change the entries in the list whose indexes are
not a multiple of 3 or 4.
• The function shall then return the sum of all entries.
*/
```

Description Example

```
package original;
import java.util.ArrayList;
import java.util.List;


class SumSquares1 {
    public static int sumSquares(List<Object> lst) {
        List<Integer> result = new ArrayList<Integer>();
        for (int i = 0; i < lst.size(); i++) {
            if (i % 3 == 0) {
                result.add((int) lst.get(i) * (int) lst.get(i));
            }
            else if (i % 4 == 0 && i % 3 == 0) {
                result.add((int) lst.get(i) * (int) lst.get(i) * (int)
            lst.get(i));
            }
            else {
                result.add((int) lst.get(i));
            }
        }

        int sum = 0;
        for (int i = 0; i < result.size(); i++) {
            sum += result.get(i);
        }
        return sum;
    }
}
```

Source Code Example

Fig. 1. Running example from the HumanEvalJava dataset

steps since it conforms to the syntax and conventions of Java and the testing framework JUnit 5. However, v_1 usually has low code coverage, as the focus at this stage is on syntactic correctness rather than test completeness. To enhance code coverage, the *Test Prefix Generation* step generates additional valid test prefixes with tentative oracles, producing an expanded test file v_2. Notably, both Steps I and II rely solely on the source code for test generation. As a result, if the source code is faulty, the tentative oracles in v_0, v_1, and v_2 may also be incorrect. To

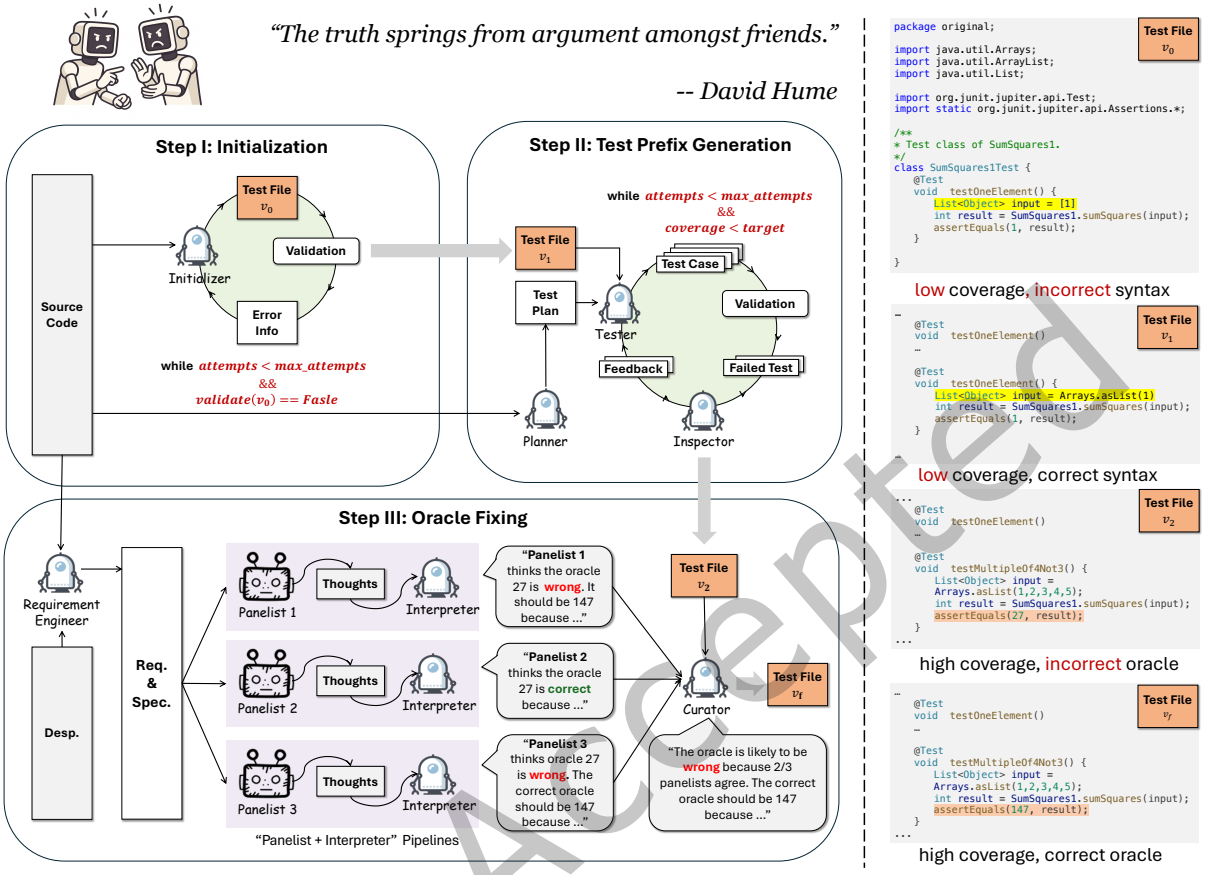

Fig. 2. Overview of CANDOR and examples. "Desp.", "Req." and "Spec." are short for "Description", "Requirement", and "Specification", respectively.

address this issue, the *Oracle Fixing* step conducts a panel discussion among multiple LLM agents to revise and correct the oracles in v_2, yielding the final test file v_f.

3.1 Step I: Initialization

This step aims to generate a syntactically correct initial test file. As shown in the first box of Figure 2, it involves two main components: (1) an LLM agent called the *Initializer* and (2) a *Validation* process. Together, they form an iterative loop to ensure the generation of a valid initial test file v_1.

Initializer. The *Initializer* is a basic LLM agent responsible for producing an initial test file v_0 based on the given source code. As illustrated in profile ① in Figure 3, the *Initializer* is guided by two types of prompts: a system prompt, which defines the model's role (e.g., as a software testing expert), and a user prompt, which provides specific inputs such as the source code, last failed test file, error messages, Java and JUnit 5 conventions and output format. The *Initializer* responds with a JSON object containing a single field, `test_file`, which holds

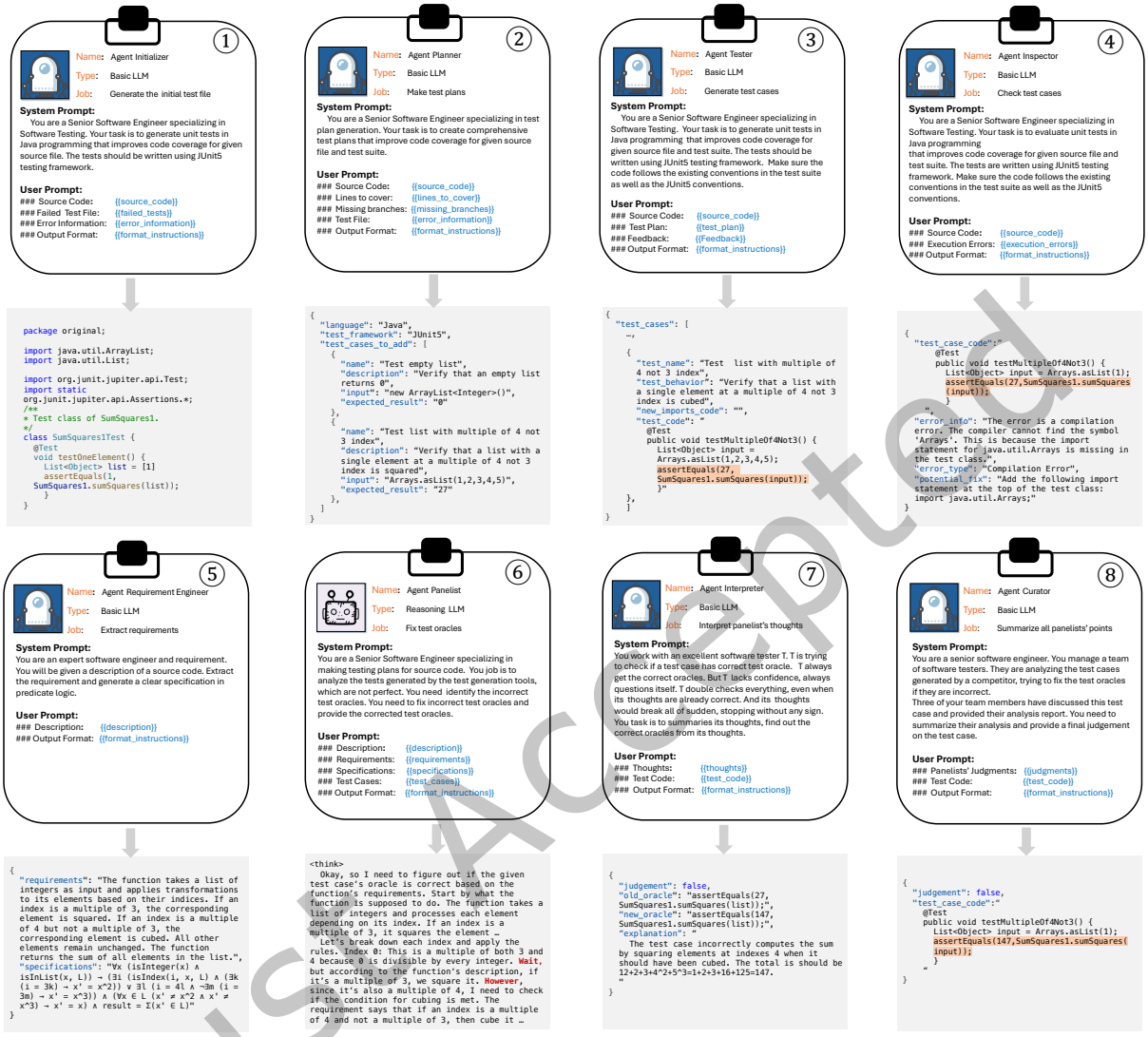

Fig. 3. Profiles of LLM agents. "Basic LLM" and "Reasoning LLM" refer to LLMs without and with reasoning capability, respectively. Each profile consists of a system prompt and a user prompt, which define the agent's role and task, respectively. Fields enclosed in double curly braces (e.g., {{...}}) in the user prompt represent variables that are dynamically filled for each specific case. The field `format_instructions` is included in all user prompts and specifies the expected output format for the agent. Below each profile, an example output is provided based on the running example. The Step I, II and III of CANDOR involve agent ①,②~④ and ⑤~⑨, respectively.

the complete generated test code. Given the source file provided in Section 2, we present the generated initialize test file v_0 under profile ① of Figure 3, including one test case `testOneElement`.

Validation. The generated test file v_0 is then passed to the *Validation* component, which attempts to compile and execute it. At this stage, v_0 often contains syntactic errors and achieves low code coverage. In the example of v_0 shown in Figure 2, the input list variable is incorrectly instantiated as `List<Object> input = [1]` following Python instead of Java conventions. If syntax-related errors are detected, the error messages are fed back to the *Initializer* for refinement. This process is repeated until a syntactically correct and error-free test file v_1 is produced or a maximum number of attempts (`max_attempts`) is reached. An example of v_1 is shown in Figure 2, where the input list variable is correctly instantiated as `List<Object> input = ArrayList.asList(1)` following Java conventions. While v_1 is syntactically valid and adheres to language and testing framework conventions, it typically still has low coverage and relies on subsequent steps to improve it.

Note that the Initialization Step is optional in CANDOR. If an initial test case is already provided, CANDOR skips Step I and proceeds directly to generating test prefixes (Step II) and oracles (Step III).

3.2 Step II: Test Prefix Generation

This step takes the initial test file v_1 from Step I and aims to enhance code coverage by generating additional test cases. As shown in the second box of Figure 2, this step involves the *Validation* process (same as in Step I) and three LLM agents: *Planner*, *Tester*, and *Inspector*. These agents collaborate in an iterative loop to propose, generate, and refine new test cases.

Planner. The *Planner* is a basic LLM agent responsible for analyzing coverage gaps and proposing new test cases accordingly. As shown in profile ② in Figure 3, the system prompt defines its role as a test plan designer tasked with improving code coverage. The user prompt requires inputs, including the source code, uncovered lines from the Jacoco coverage report, and the current test file. Based on this information, the Planner outputs a list of structured test plans in JSON format under the field `test_cases_to_add`, with each test case described by a `name`, a brief natural language `description`, `input`, and `expected output`. A snippet of a generated plan based on the running example is provided in profile ②, Figure 3. This plan proposes two test cases to enhance code coverage: `Test empty list` and `Test list with multiple of 4 not 3 index`.

Tester. The *Tester* is a basic LLM agent that generates executable test code based on the plans proposed by the *Planner*. As shown in profile ③ in Figure 3, the system prompt defines the *Tester*'s role as a Java unit test generator following JUnit 5 and the conventions of the given initial test file. The user prompt includes the source code, the test plan from the *Planner*, and feedback from the *Inspector* (if any). The output is a JSON object containing a list of `generated_test_cases`, with each entry specifying the `behavior`—a natural language explanation of what the test is verifying, `test_name`, `test_code`, and any `new_import_statements` required to execute this test case. A snippet of the generated tests based on the running example is provided under profile ③ in Figure 3. The *Tester* generates the test case `testMultipleOf4Not3`, with a test prefix of `ArrayList.asList(1,2,3,4,5)` and a tentative oracle `assertEquals(27, SumSquares1.sumSquares(input))`. Notice the tentative oracle is incorrect: it is calculated as $1 + 2 + 3 + 4^2 + 5 = 27$, whereas the correct oracle should be $1 + 2 + 3 + 4^2 + 5^3 = 147$. The fifth element was not cubed as specified in the *description* due to a bug in the *source code*, specifically the conditional statement `if (i%4==0&&i%3==0)` (Figure 1). Additionally, the *Tester* fails to identify the necessary `import` statements for `ArrayList`.

Inspector. The *Inspector* is a basic LLM agent that evaluates the compiled and executed test cases to detect and explain any errors. Its role, as shown in profile ④ in Figure 3, is to inspect the last generated test code and its execution result and provide feedback for further improvement. The user prompt includes the source code and error messages from the validation process. The output consists of structured feedback entries, each containing the `failed_test_code`, the `error_message`, the `error_type`, and a `potential_fix`. This feedback is then sent back to the *Tester* for refinement. A snippet of the generated feedback based on the running example is presented

under profile ④ of Figure 3. In this case, the *Inspector* correctly identifies the missing `import` statements for `ArrayList` in the test file v_1. However, it does not flag the incorrect oracle, as the analysis is performed against the faulty *source code*, which exhibits the same buggy behavior as the test output.
Step II is iterative; the *Planner* suggests additional test cases, the *Tester* generates code, and the *Inspector* provides corrective feedback. The loop terminates either when the maximum number of attempts is reached or when the code coverage satisfies a predefined threshold. This step produces the test file v_2, which tends to have significantly higher coverage compared to v_0 and v_1, while maintaining syntactic correctness.

3.3 Step III: Oracle Fixing

The first two steps generate test cases and tentative oracles directly from the source code. However, when the source code is faulty, the tentative oracles may also be incorrect. To address this limitation, Step III focuses on fixing the oracles using natural language *descriptions*, which better reflect the intended functionality. As shown in Figure 2, this step involves a *Requirement Engineer* agent, multiple dual-LLM pipelines (*Panelist + Interpreter*), and a *Curator* agent. These agents collaborate to produce more accurate test oracles, even in the presence of incorrect code implementation.

Requirement Engineer. The *Requirement Engineer* is a basic LLM agent responsible for extracting both natural language requirements and formal specifications from the provided description of the function. As illustrated in profile ⑤ in Figure 3, the system prompt defines its role as a software engineer specializing in requirement analysis. The user prompt includes the textual description of the code, and the output consists of a set of human-readable requirements (e.g., input type and expected behavior). When possible, it also produces a formal specification expressed in predicate logic. The grammar follows standard predicate logic and, if a formalization is not applicable or too ambiguous for the task, the specification is left empty, and only the natural language requirements are used for downstream oracle correction. The *Requirement Engineer* Agent attempts to extract the specifications for every method under test and decides by itself on whether or not to provide them. The extracted requirements and specifications for the running example are shown under profile ⑤ in Figure 3.

Dual-LLM pipelines. Each pipeline comprises a reasoning LLM agent *Panelist* and a basic LLM agent *Interpreter*. Each *Panelist* functions as an independent oracle corrector. As shown in profile ⑥ in Figure 3, the system prompt defines its role as a testing expert tasked with identifying and fixing incorrect test oracles generated by automated tools. The user prompt requires the natural language description, the extracted requirements and specifications from the *Requirement Engineer*, and the set of test cases generated in Step II. Each agent analyzes the given test case and determines whether the existing oracle is correct. If not, it provides a corrected oracle based on the intended behavior described in the requirements and specification. To mitigate the "overthinking phenomenon" (see Section 1) of reasoning LLMs, each *Panelist* is paired with a basic LLM agent *Interpreter*. Each *Interpreter* is responsible for extracting relevant insights from *Panelist*'s verbose thoughts and producing structured evaluations of the tentative oracles. As shown in profile ⑦ of Figure 3, the system prompt defines its role as an assistant working with an excellent but self-doubting tester. The user prompt requires the *Panelist*'s thoughts and the test code under examination. Example outputs of both *Panelist* and *Interpreter*, based on the running example, are shown under profiles ⑥ and ⑦ of Figure 3, respectively. In this case, the *Panelist* generates a lengthy, self-questioning reasoning process — using phrases like `Wait` and `However` — while the *Interpreter* successfully extracts concise and structured oracle evaluations.

Curator. Multiple dual-LLM pipelines operate in parallel, each making independent judgments of the tentative oracle. The *Curator* aggregates these judgments and determines the final oracle. As shown in profile ⑧ of Figure 3, the system prompt defines its role as a leader of a software engineering team. While team members discuss the correctness of the tentative oracle, the *Curator* is responsible for summarizing and issuing a final judgment. Its

user prompt requires individual judgment from each pipeline and the test code under examination. An example output of *Curator* based on the running example is shown below profile ⑧ of Figure 3. In this case, the *Curator* successfully identifies the tentative oracle as incorrect and revises it into the correct oracle `assertEquals(147, SumSquares1.sumSquares(input));`.

Step III yields the final test file v_f, which not only achieves high code coverage—like v_2—but also contains corrected oracles based on the natural language *description*.

4 Experimental Setup

To assess CANDOR, we ask the following research questions (RQs):

RQ.1 How well does CANDOR perform at generating high coverage test prefixes? How does its performance vary under different levels of code complexity?

RQ.2 Can CANDOR produce accurate test oracles, both when the source code is correct (i.e., conforms to the intended requirements) and when it is faulty?

RQ.3 What is the individual effectiveness contribution of key components in CANDOR, including the *Planner*, the *Requirement Engineer*, and the panel discussion?

RQ1 evaluates the overall effectiveness of the generated test cases by comparing code coverage and mutation scores with state-of-the-art baselines. We conduct experiments on the benchmark HumanEvalJava dataset [8], which is extensively studied in the literature [24, 44, 46, 48, 68, 74, 86]. Additionally, we constructed a new dataset LeetCodeJava from LeetCode Platform [2] to assess CANDOR's robustness across different levels of code complexity. For RQ2, we focus on the accuracy of oracle generation. We evaluate oracle correctness under two conditions: (1) when the source code is correct and conforms to the intended requirements, and (2) when the code is faulty. To simulate faulty scenarios, we use a commonly used tool (PiTest [3]) to inject mutations into the source code of both HumanEvalJava and LeetCodeJava to investigate CANDOR's robustness and its practical utility in real-world testing scenarios. Specifically, we generated three mutants for each SUT. With RQ3, we assess the individual contributions of key components within CANDOR through a series of ablation studies. Each ablation study compares CANDOR with a variant that omits one key component, including the *Planner* (w/o Planner), *Requirement Engineer* (w/o Req.), and the panel discussion (w/o Panel and w/ Voting). In particular, "w/o Panel" retains only one Panelist and removes the Curator for oracle generation. "w/ Voting" replaces the panel discussion with a simple majority-voting mechanism, removing the Curator from CANDOR. Instead of the *Curator* reasoning over and summarizing the arguments of multiple *Panelists*, the final oracle is directly selected based on majority agreement. We do not perform ablations on other agents, as they are essential for generating valid test cases; removing them would render CANDOR non-functional. To mitigate the influence of randomness, all the experiments are repeated three times.

4.1 Datasets

In this work, we focus on unit test generation for Java. But at this stage, we consider only Java methods that do not depend on user-defined classes or external libraries, thus excluding datasets such as Defects4J [45] and SF110 [29]. Addressing such dependencies requires additional techniques, such as retrieval-augmented generation (RAG) and mocking, which we plan to address in the future. As the first multi-agent-based framework for Java projects, CANDOR lays the foundation of testing Java methods using only off-the-shelf LLMs, a significant challenge in itself.

Specifically, we evaluate CANDOR on two representative datasets: HumanEvalJava [8] and LeetCodeJava. HumanEvalJava is a representative dataset for Java unit test generation, actively studied by recent researches [13, 22, 24, 35, 44, 46, 48, 68, 74]. Further, we constructed a more challenging dataset, LeetCodeJava, to evaluate CANDOR under scenarios with different complexity. To demonstrate the representativeness of our selected

datasets, we report key statistics in the remainder of this section, including the number of programs under test, the average line of code, and cyclomatic complexity (CC). We remark that CC is a well-established measure of control-flow complexity, quantifying the number of linearly independent execution paths in a method [9, 23, 42, 73]. Higher CC implies more branches to explore, thus making both test prefix and oracle generation more challenging. According to a large-scale empirical study of more than 862,517 Java classes from 1,000 open GitHub repositories [69], the average CC of Java classes is 5.46.

HumanEvalJava. It consists of 160 Java programs implemented by Siddiq et al. to solve problems introduced in [8]. Each program defines a Java class under test containing a single method. The length of these programs ranges from 20 to 222 lines, with an average of 41 lines of code. The average CC of HumanEvalJava methods is 4.90, which is close to the average CC for Java methods (5.46) reported in [51]. This indicates that HumanEvalJava exhibits comparable control-flow complexity and is representative of typical Java methods.

LeetCodeJava. The LeetCode Platform lists over 3,000 programming problems, categorized by difficulty levels: “easy”, “medium”, and “hard” [2]. More difficult problems generally require more complex code. Due to time and computational resource constraints, we randomly sampled 50 programming problems from the ”medium” and ”hard” difficulty categories, for a total of 100. Then, we construct the LeetCodeJava dataset by collecting their corresponding solutions [21]. These solutions are considered correct implementations because they are actively maintained and validated by the community. In addition to these practical considerations, we specifically chose programs of greater difficulty to ensure the dataset included more complex structural characteristics, thereby providing a more rigorous evaluation of our approach. These methods have an average of 33 lines of code, ranging from 7 to 126. The average number of lines of code for the “Medium” and “Hard” categories is 31 and 39, respectively. The average CC of methods in LeetCodeJava is generally higher than that of HumanEvalJava (4.90), with CC values for LeetCode-Medium and LeetCode-Hard reaching 6.30 and 9.46, respectively. These statistics highlight that LeetCodeJava-Medium includes Java classes with CC levels comparable to those observed in real-world code (5.46) [69], making it representative, while LeetCodeJava-Hard also contains more challenging cases with higher CC values.

4.2 Baselines

We compare CANDOR against four representative baselines: EvoSuite [28], LLM-Empirical [68], TOGLL [39] and EVO-CANDOR. To the best of our knowledge, CANDOR is the first end-to-end, multi-agent-based approach for Java test generation that jointly addresses both test prefix and specification-based oracle generation. Among the baselines, LLM-Empirical is the most closely related, as it also performs end-to-end test generation but relies on a single off-the-shelf LLM. In contrast, EvoSuite and TOGLL represent state-of-the-art techniques for test prefix generation and specification-based oracle generation, respectively.

EvoSuite. This widely adopted automated test-generation tool for Java programs uses search-based techniques, such as genetic algorithms, to systematically generate unit tests. It analyzes the program under test to generate test cases that maximize code coverage metrics, such as line and branch coverage. In addition to generating test prefixes, EvoSuite automatically produces regression oracles—assertions that capture the current program behavior by checking expected outputs, exceptions, and state changes. These generated tests are structured as JUnit test cases, making EvoSuite a popular baseline for research in automated testing and regression testing [28, 39, 76]. As recently reported [72], EvoSuite remains the SOTA approach in generating test prefixes in terms of code coverage, even when compared to the latest LLM-based approaches. EvoSuite thus serves as a baseline for test prefix generation in this work.

LLM-Empirical. The LLM-Empirical method, proposed by Siddiq et al. [68], leverages LLMs such as Codex, GPT-3.5-Turbo, and StarCoder to generate JUnit tests for Java programs using prompt engineering without fine-tuning. Unlike EvoSuite, which generates regression oracles based on the current implementation behavior, LLM-Empirical generates test oracles based on natural language descriptions of the SUT. It represents the SOTA in prompt-based, LLM-driven oracle generation for Java unit tests. Among the models used in LLM-Empirical, Codex (specifically, code-davinci-002) and GPT-3.5-Turbo are the most capable. Moreover, code-davinci-002 is no longer publicly available. As a result, we implement LLM-Empirical using GPT-3.5-Turbo. Other advanced LLMs cannot be directly applied to LLM-Empirical because their implementation—particularly the component that fixes test cases—is specifically tailored to the behavior and outputs of the LLM they use. EvoSuite serves as a baseline for both test prefix and oracle generation in this work.

TOGLL. Recently, Hossain and Dwyer [39] prensented TOGLL, fine-tuning LLMs on the SF110 dataset to generate specification-based oracles. Experimental results demonstrate that TOGLL outperforms prior oracle generators by large margins in terms of oracle correctness, bug-detection effectiveness, and mutation score, setting a new SOTA in JUnit oracle generation. Additional experiments show that TOGLL achieves optimal performance by fine-tuning a CodeParrot LLM with a prompt template that includes both program descriptions and method signatures. However, since method signatures are neither commonly available in practice nor in our experimental datasets, we do not include them in the prompt. Specifically, the prompt templates include a program description, program source code, and a test prefix generated by EvoSuite. We note that comparing TOGLL and CANDOR is biased against the latter, as TOGGL is fine-tuned on external datasets, whereas CANDOR uses off-the-shelf, general-purpose LLMs. Nevertheless, we list TOGLL as a baseline in this work, given its SOTA performance in the oracle generation domain, while EvoSuite serves as a baseline for test oracle generation.

EVO-CANDOR. TOGLL generates oracles based on EvoSuite test prefixes, while CANDOR conducts end-to-end generation, with both test prefixes and oracles generated by LLMs. Oracles for different test prefixes are not directly comparable. To ensure a fair comparison of TOGLL and CANDOR in terms of oracle quality (RQ2), we consider a variant of CANDOR, EVO-CANDOR, which generates test oracles based on the identical EvoSuite test prefixes used by TOGLL.

4.3 Evaluation Metrics and Statistical Testing

We evaluate the effectiveness of CANDOR for test generation in terms of code coverage, mutation score, and oracle correctness. The coverage and mutation scores are calculated only for test prefixes with regression oracles to ensure a fair comparison with EvoSuite. To reduce the influence of randomness, we repeat all the experiments three times and calculate the average of each metric. We also evaluate the significance of differences between CANDOR and baselines by conducting statistical tests. Specifically, we compare these metrics between CANDOR and each baseline across programs using a paired non-parametric statistical test (i.e., Wilcoxon Signed Rank test).

Code coverage. This metric measures the proportion of code executed by passing tests in the generated test suites, typically reported as line coverage and branch coverage. Line coverage refers to the percentage of lines that are executed by the test suite, while branch coverage measures the percentage of control flow branches (e.g., if/else conditions) that are reached. Code coverage is a widely used metric for assessing the effectiveness of test prefixes, but it does not address test oracles [38].

Mutation score. This calculates the percentage of synthetic bugs (mutants) detected by the test suite. We use PiTest with its standard mutation operators to generate mutants for each program under test. A mutant is considered killed if at least one test in the suite fails when executed against the mutated program. The mutation

score is calculated as the percentage of killed mutants over the total number of generated mutants. A higher mutation score indicates a more fault-revealing test suite.

Oracle correctness. This metric measures the accuracy of the assertions (oracles) in a generated test suite. Specifically, it is calculated as the number of correct oracles divided by the total number of oracles in the test suite. An oracle is considered correct if it accurately reflects the intended behavior of the program as specified by its natural language description or reference implementation. Specifically in our experiments, both HumanEvalJava and LeetCodeJava are well-established projects with no known bugs. Therefore, any failure of a generated test oracle suggests it is incorrect.

Statistical testing. To compare CANDOR against baselines, we perform the Wilcoxon Signed Rank test for all RQs, as recommended in [6], with a significance level of 0.05. The Wilcoxon Signed Rank test is a non-parametric statistical test used to determine whether there is a significant difference between two independent distributions (e.g., method A vs. method B). For RQ2, we also report the A_{12} effect size to measure the magnitude of difference in oracle correctness between CANDOR and LLM-Empirical. A_{12} ranges between 0 and 1, representing the probability that method A yields better results than method B.

4.4 Implementation

All experiments are conducted on a Precision 7960 Tower XCTO workstation equipped with an Intel Xeon w9-3495X processor and dual NVIDIA RTX 6000 Ada GPUs. The implementation is written in Python, using the LangChain library [1] for LLM integration.

Most works in the literature use EvoSuite as a baseline with its default, recommended configuration ("`assertion_strategy=``mutation''; assertion_timeout=60s`") [14, 39, 68, 76, 86]. However, to ensure EvoSuite is fully exercised, we experimented with different time budgets (`1`, `2`, `3`, `4`, `5`, `10`, `30`, `60`, and `120` minutes per SUT) and assertion strategies ("`MUTATION`", "`ALL`", and "`UNIT`"). We find the resulting coverage was not further improved after "`assertion_timeout > 2 min`", stuck at a plateau as previously reported [47]. The choice of assertion strategies did not significantly affect the quality of the generated tests. Therefore, we utilize EvoSuite with the default assertion strategy (`assertion_strategy=``mutation''`) and an increased timeout threshold (`assertion_timeout=2 min`). For PiTest, we employ the default group of mutation operators following the common practice in the literature [19, 64, 78].

We use Llama 3.1 70B as the basic LLM and DeepSeek R1 Llama-distilled 70B as the reasoning LLM. DeepSeek R1 is selected for its strong reasoning capability among open-source models. For the basic LLM, we also experimented with other commonly used alternatives such as CodeLlama 70B and Mistral 22B, however, Llama 3.1 70B achieved the best overall performance. The corresponding results are reported in the Appendix for completeness. The max_attempt parameter in the Initialization and Test Prefix Generation step is set to 3. If CANDOR fails to generate an initial test file for a given method under test within these attempts, it stops further trials and proceeds to the next method. We limit the number of output tokens of DeepSeek to 2000 for efficiency. The number of "Panelist + Interpreter" pipeline is set to be 3. We experimented with 1 to 5 pipelines. However, the improvements beyond 3 are not significant and incur much longer generation time. We release our code and data publicly to facilitate replication[1].

[1]https://github.com/qiqiguana/candor

5 Experiment Results

5.1 RQ1 Results: Test Prefix Quality

Table 1 presents the experimental results for RQ1, comparing CANDOR with LLM-Empirical and EvoSuite in terms of test prefix quality, measured by line coverage ("Line"), branch coverage ("Branch"), and mutation score ("Mutation").

Table 1. Experimental results of RQ1. "*" denotes CANDOR achieves significantly better results than the compared baseline.

	HumanEvalJava			Leetcode-Medium			Leetcode-Hard		
	Line	Branch	Mutation	Line	Branch	Mutation	Line	Branch	Mutation
LLM-Empirical	0.704*	0.688*	0.823* (2011/2443)	0.771*	0.732*	0.868* (530/611)	0.714*	0.729*	0.778* (668/859)
EvoSuite	0.961*	0.942	0.858* (2096/2443)	0.959*	**0.959**	0.845* (516/611)	0.984	0.976	0.888* (763/859)
CANDOR	**0.991**	**0.950**	**0.980** (2384/2443)	**0.990**	0.949	**0.939** (574/611)	**0.989**	**0.980**	**0.937** (805/859)

CANDOR consistently achieves high *line coverage* across all datasets, with the lowest value still reaching 0.989 on the Leetcode-Hard dataset. Compared to EvoSuite, CANDOR shows marginal improvements, with a maximum gain of 0.031 (0.990 − 0.959 on Leetcode-Medium). In contrast, LLM-Empirical performs substantially worse than CANDOR across all three datasets, with the largest gap reaching 0.287 (0.991 − 0.704) on HumanEvalJava. Wilcoxon Signed Rank tests confirm that the differences between CANDOR and LLM-Empirical are statistically significant (p-value $<$ 1e−4), whereas the differences between CANDOR and EvoSuite are not statistically significant (p-value $>$ 0.05).

For *branch coverage*, both CANDOR and EvoSuite substantially outperform LLM-Empirical. The maximum branch coverage achieved by LLM-Empirical is 0.732 on Leetcode-Medium, whereas the minimum branch coverage achieved by EvoSuite or CANDOR is 0.942 on HumanEvalJava. Wilcoxon Signed Rank tests confirm that the differences between CANDOR and LLM-Empirical are statistically significant (p-value $<$ 1e−4), whereas the differences between CANDOR and EvoSuite are not statistically significant (p-value $>$ 0.05).

For *mutation score*, CANDOR performs the best across all three datasets, successfully killing 2384 (out of 2443), 574 (out of 611), and 805 (out of 859) mutants on HumanEvalJava, Leetcode-Medium, and Leetcode-Hard, respectively. CANDOR outperforms EvoSuite by at least 0.049 (0.937−0.888 on Leetcode-Hard) and LLM-Empirical by at least 0.157 (0.980 − 0.823 on HumanEvalJava). Wilcoxon Signed Rank tests confirm that all differences between CANDOR and the baselines are statistically significant (p-value $<$ 1e−4). We hypothesize that the advantage of CANDOR in mutation scores stems from the LLM's semantic understanding. Unlike EvoSuite, which primarily evolves test prefixes to satisfy structural coverage criteria, CANDOR also considers the semantics of the program under test (e.g., respecting input constraints, maintaining data invariants, and producing realistic usage patterns). These semantically meaningful test prefixes are more effective at revealing subtle behavioral deviations introduced by mutants, even when overall coverage is similar. Consequently, CANDOR achieves substantially higher mutation scores than EvoSuite.

We also observe that CANDOR's performance remains stable as the dataset shifts from Leetcode-Medium to Leetcode-Hard, with only slight drops in line coverage (−0.01) and mutation score (−0.02), indicating robustness to increasing problem difficulty. We posit that this robustness originates from the pretraining of LLMs on a vast and diverse corpus, which allows CANDOR to generalize well across programs of varying complexity in this context.

Overall, CANDOR outperforms both baselines across all metrics, with one exception: on the Leetcode-Medium dataset, EvoSuite achieves slightly higher branch coverage than CANDOR by a margin of 0.01. However, the

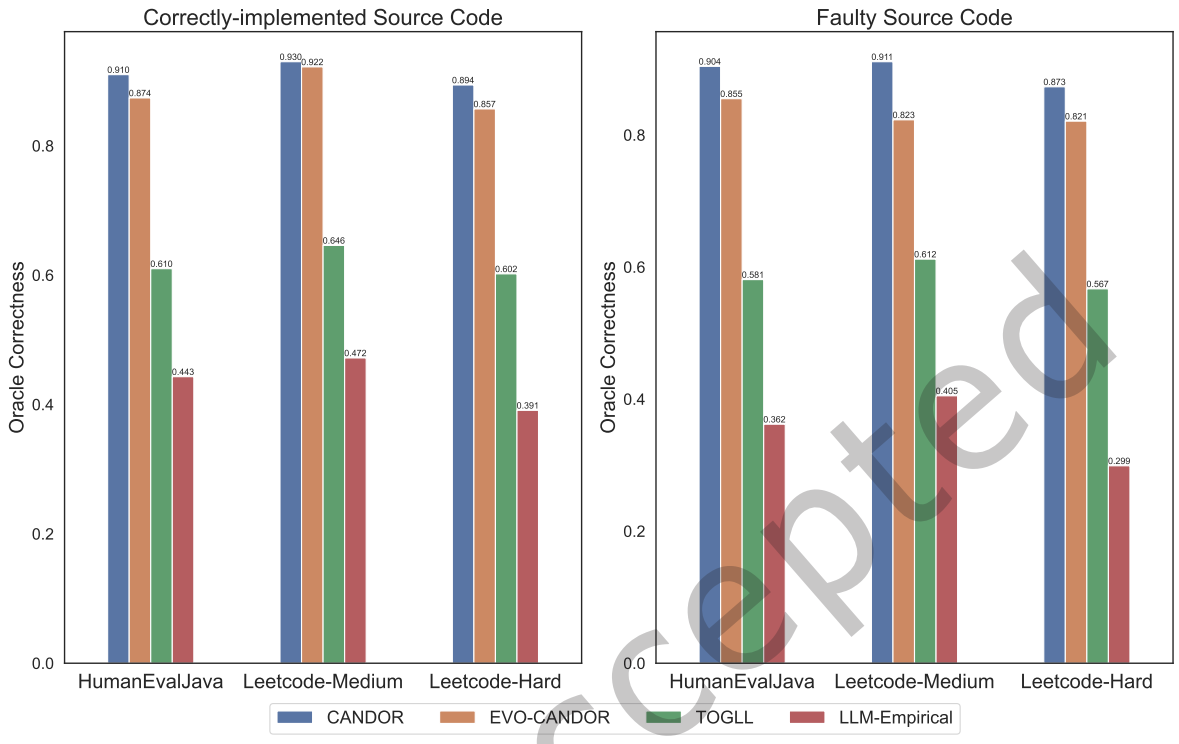

Fig. 4. RQ2 results: Comparison of CANDOR, TOGLL and LLM-Empirical on correctly-implemented and faulty source code in terms of oracle correctness.

differences between CANDOR and EvoSuite in terms of line and branch coverage are marginal and mostly insignificant, indicating that CANDOR achieves comparable performance in code coverage to EvoSuite. Moreover, CANDOR significantly outperforms EvoSuite in mutation scores, suggesting better bug-detection capability.

> Overall, CANDOR and EvoSuite achieve comparable, high-quality test prefixes across all datasets in terms of line coverage and branch coverage. However, CANDOR significantly outperforms EvoSuite in mutation scores, with improvements exceeding 0.049 on all datasets.

5.2 RQ2 Results: Test Oracle Quality

Figure 4 compares the correctness of oracles generated by CANDOR, LLM-Empirical, and TOGLL. We exclude EvoSuite from this comparison since it can only produce regression oracles rather than specification-based oracles.

On Correct Code. As depicted in the left part of Figure 4, CANDOR achieves the highest oracle correctness scores of 0.910, 0.930, and 0.894 on HumanEvalJava, Leetcode-Medium, and Leetcode-Hard, respectively. The gap between CANDOR and LLM-Empirical is substantial, with a minimum of 0.467 ($0.910 - 0.443$) on HumanEvalJava

in terms of oracle correctness. When compared to TOGLL, we refer to EVO-CANDOR's results since they both use the same EvoSuite test prefixes. We observe that, despite the slight performance decrease compared to CANDOR, EVO-CANDOR still outperforms TOGLL by a large margin, with a minimum of 0.255 (0.857 − 0.602) on Leetcode-Hard. We conduct the Wilcoxon Signed Rank tests for all the comparisons, and the results confirm that all differences are statistically significant (p-value $<$ 1e−4). We further calculate Vargha and Delaney's effect size for the comparison between EVO-CANDOR and TOGLL and found a strong effect size ($A_{12} = 0.920$), indicating there is a 92 % chance that EVO-CANDOR can outperform TOGLL.

We posit that the superior performance of CANDOR can be explained by its capability to generate oracles based on only informal descriptions. In contrast, the optimal configuration of TOGLL requires not only the descriptions but also method signatures, which are often unavailable in real-world projects and are absent in our datasets. Moreover, TOGLL is fine-tuned on the SF110 dataset, consisting of tests generated with EvoSuite v1.0.6, Java 8, and Ant Builder. Program features beyond this environment cannot be recognized, thereby diminishing the effectiveness of TOGLL. For instance, we observe that TOGLL struggles with newer Java features, such as "`var`" for type inference and lambda expressions such as "`list.stream().map(String::toUpperCase).toList();`"

On Faulty Code. As described in Section 4, the faulty code is created by PiTest, which introduces mutations into correctly-implemented benchmarks: HumanEvalJava, Leetcode-Medium, and Leetcode-Hard. As shown in the right part of Figure 4, CANDOR remains the best approach across all datasets. Compared to LLM-Empirical, the gaps are even larger than those in correctly-implemented code in terms of oracle correctness, with a minimum of 0.506 (0.911 − 0.405) on Leetcode-Medium. When comparing with TOGLL, EVO-CANDOR outperforms it by a large margin, with a minimum of 0.211 (0.823 − 0.612) on Leetcode-Medium. Wilcoxon Signed Rank tests confirm that all differences in the comparisons between CANDOR and LLM-Empirical, and between EVO-CANDOR and TOGLL, are statistically significant (p-value $<$ 1e−4). We also observe an even stronger Vargha and Delaney's effect size ($A_{12} = 0.960$) in the comparison between EVO-CANDOR and TOGLL, as there is a 96 % chance for EVO-CANDOR to outperform TOGLL.

The superior performance of CANDOR on faulty code highlights its fault-detection capability. We believe such performance is achieved by deriving oracles from natural language descriptions of the source code in an effective manner, rather than relying solely on the source code itself. Specifically, the panel discussion employed by the CANDOR enables the cross-validation on the generated oracles. This design prevents hallucination, yielding more accurate oracles than approaches that rely on single-agent reasoning.

Again, it is important to recall that comparing CANDOR with TOGLL is somewhat unfair, as TOGLL is fine-tuned with an extra dataset, while CANDOR uses off-the-shelf LLMs. Despite this advantage, CANDOR still outperforms TOGLL substantially while offering greater flexibility. Indeed, unlike TOGLL, CANDOR is flexible when applied to new Java versions or system environments, eliminating the need for fine-tuning on a version-specific dataset. This is particularly important in practice as many software systems and environments are frequently upgraded, making fine-tuning costly and time-consuming.

> CANDOR generates significantly more accurate test oracles than TOGLL across all datasets, even though TOGLL is fine-tuned with additional data. Moreover, CANDOR demonstrates robustness to faulty code by leveraging code descriptions.

5.3 RQ3 Results: Ablation study

Table 2 compares CANDOR with three ablation configurations: removing the *Planner* agent ("w/o Planner"), removing the *Requirement Engineer* agent ("w/o Req."), and removing the panel discussion ("w/o Panel").

Table 2. Experimental results of RQ3. "N/A" indicates the current ablation configuration has no influence on this metric; therefore, the corresponding experiments are not conducted. "*" denotes CANDOR achieves significantly better results than the compared baseline.

	HumanEvalJava				Leetcode-Medium				Leetcode-Hard			
	Line	Branch	Mutation	Oracle	Line	Branch	Mutation	Oracle	Line	Branch	Mutation	Oracle
CANDOR	**0.991**	**0.970**	**0.980** (2384/2443)	**0.910**	**0.990**	**0.949**	**0.939** (574/611)	**0.930**	**0.989**	**0.980**	**0.937** (805/859)	**0.894**
w/o Planner	0.892*	0.840*	0.869* (2125/2443)	N/A	0.940*	0.903*	0.869* (531/611)	N/A	0.935*	0.922*	0.833* (716/859)	N/A
w/o Req.	N/A	N/A	N/A	0.882*	N/A	N/A	N/A	0.923	N/A	N/A	N/A	0.873*
w/o Panel	N/A	N/A	N/A	0.824*	N/A	N/A	N/A	0.855*	N/A	N/A	N/A	0.827*
w/ Voting	N/A	N/A	N/A	0.896*	N/A	N/A	N/A	0.910*	N/A	N/A	N/A	0.877*

Removing the *Planner* significantly reduces test prefix quality across all datasets. The minimum decreases are observed on Leetcode-Medium with line coverage, branch coverage, and mutation score dropping by at least 0.050 (0.990 − 0.940), 0.046 (0.949 − 0.903), and 0.070 (0.939 − 0.869), respectively. Wilcoxon Signed Rank tests confirm these decreases are statistically significant (p-value $<$ 1e−4). This underscores the critical role of the *Planner*, which analyzes current coverage and strategically plans to cover missing lines. In our experiments, we observe that without the *Planner*, CANDOR generates tests exploring easy code paths in most cases, lacking a coherent strategy to explore untested code paths.

Removing the *Requirement Engineer* or the panel discussion primarily affects oracle correctness. In the absence of the *Requirement Engineer*, oracle correctness decreases slightly by 0.028 (0.910−0.882), 0.007 (0.930−0.923), and 0.021 (0.894 − 0.873) on HumanEvalJava, Leetcode-Medium, and Leetcode-Hard, respectively. Wilcoxon Signed Rank tests indicate the changes on HumanEvalJava and Leetcode-Hard are statistically significant (p-value $<$ 1e−3), whereas the decline on Leetcode-Medium is not significant (p-value = 0.17). This likely suggests that the descriptions of source code in these three datasets are clear and well-structured, which reduces the need for explicit requirement parsing using the *Requirement Engineer* agent. However, in real-world scenarios with less clear documentation, this agent is expected to provide greater benefits.

In contrast, removing the panel discussion ("w/o Panel") causes substantial and statistically significant drops in oracle correctness, i.e., 0.086 (0.910 − 0.824), 0.075 (0.930 − 0.855), and 0.067 (0.894 − 0.827) on HumanEvalJava, LeetCodeJava-Medium, and LeetCodeJava-Hard, respectively. Replacing the panel discussion with a majority voting mechanism ("w Voting") also leads to consistent decreases across all datasets in terms of oracle correctness, with a minimum of 0.014 (0.910 − 0.896) on the HumanEvalJava dataset. We performed Wilcoxon tests and found that all decreases are statistically significant ($pvalue < 1e-2$). This aligns with our expectations, as without the panel discussion, LLMs are more prone to uncertainty and hallucination, generating plausible but incorrect oracles more frequently. In our experiments, we observed serious hallucinations during the panel discussion; in over 70 % of the cases, the *Panelists*' discussions contained clear disagreements. For example, a method "`max_element()`" that computes the maximum of an array was mistakenly used to compute the minimum instead by one or more *Panelists* (out of three). However, the *Curator* can correct these errors by cross-checking the reasoning logic, inputs, and outputs of each *Panelist* rather than relying solely on majority voting. This design ensures that incorrect or inconsistent outputs are identified and corrected, making the panel discussion more robust than simple majority voting in producing accurate oracles.

Ablation results confirm the essential role of the *Planner* in producing high-quality test prefixes and of the panel discussion in generating accurate test oracles. The *Requirement Engineer* generally contributes to improving oracle accuracy, with its impact expected to be greater when the quality of the natural language description is low.

6 Discussion

Despite the high effectiveness of LLMs for unit test generation, instructing them to act as expected remains challenging. In this section, we share several key insights from our experiments on effective LLM instruction (Section 6.1), including adopting specialized agents, strategies to mitigate LLM hallucination, and methods to handle the verbosity of reasoning LLMs. We also report the limitations of our method in Section 6.2.

6.1 Key Insights

Simplifying Tasks via Specialized LLM Agents. LLMs are increasingly studied not only due to their effectiveness but also their ease of use, as many tasks can be handled by crafting prompts for one single LLM [34]. However, we found that these single-LLM approaches fall short on complex tasks, such as unit test generation. Unit test generation involves several subtasks, including planning and generating test cases. When an LLM is asked to perform both tasks simultaneously, it often becomes confused about its roles. For example, we observed that when tasked with both planning and generating Java tests, the LLM sometimes produced syntax like `[1]` instead of the correct Java syntax `Arrays.asList(1)`. We hypothesize that this happens due to interference from the planning sub-task, which is often expressed in natural language or in weakly typed languages like Python in LLMs' pretraining corpora. The LLM blends conventions of planning and Java test case generation, resulting in a syntactically wrong test case. By decomposing complex tasks and assigning each sub-task to a specialized LLM agent, we prevent this interference and improve the effectiveness of CANDOR in unit test generation.

Mitigating Hallucination via a Panel Discussion. LLMs are notoriously prone to hallucination, generating plausible yet nonfactual content [41]. We found that hallucination is particularly frequent in test-oracle generations, where LLMs produce incorrect oracles even when clear instructions are provided. For instance, when told to `square the integer if its index is a multiple of 3`, an LLM may incorrectly leave the number unchanged. To address this, we use multiple LLMs to independently generate oracles and then compare their outputs. Since hallucinations are usually inconsistent, taking a consensus across models helps filter them out. Instead of simple majority voting, we allow the LLMs to explain their reasoning in a panel discussion and determine the answer based on the most consistent logic.

Stopping Overthinking via a Dual-LLM Pipeline. Reasoning LLMs like DeepSeeek R1 are effective but often overly verbose, producing lengthy outputs that significantly increase generation time. In our experiment, using DeepSeek directly for oracle evaluation sometimes produced outputs exceeding 10,000 tokens, taking hours to complete a single test file. In fact, we observed that DeepSeek often identified correct oracles early. Still, the model would continue second-guessing itself by thinking "`I think the correct oracle should be 147. But, wait, maybe I made a mistake ...`". To prevent this "overthinking", we truncate DeepSeek's output at 2,000 tokens and use a basic LLM to extract the correct oracle from its reasoning. This dual-LLM pipeline preserves the benefit of deep reasoning while keeping outputs concise.

6.2 Threats to validity

Construct Validity. We evaluate CANDOR using code coverage, mutation score, and oracle correctness. While real bug detection on datasets like Defects4J is another useful metric, we did not include it because such project-level datasets are beyond the scope of this work, as discussed earlier. Instead, we use the mutation score as a proxy for bug-finding capability.

Internal Validity. Our results may be influenced by the choice of LLMs used in the framework. We use LLaMA 3.1 70B as the basic LLM and DeepSeek R1 as the reasoning LLM, selected based on preliminary experiments for their stability and open-source availability. While other popular models, such as GPT, Grok, and Mistral, exist, exhaustively evaluating all LLMs is beyond the scope of this work. The choice of LLMs may affect performance, but we aim to demonstrate the potential effectiveness of our method rather than benchmark specific models, which might indeed yield even better results. Another potential threat is data leakage, where our datasets may be included in the LLMs' pretraining data, leading to inflated performance. To mitigate this, we evaluate the mutation score, which detects behavioral changes in synthetically modified programs. These mutated versions are highly unlikely to appear in pretraining corpora, providing a more reliable evaluation of CANDOR.

Conclusion Validity. A potential threat is the randomness in LLM outputs, which may affect the consistency of results. Specifically, LLMs can produce different outputs even when given identical inputs. To mitigate this, we repeat all experiments three times, compute the average, and apply statistical testing to assess the significance of differences in averages across programs.

External Validity. One threat is that our evaluation is conducted on only two datasets (i.e., HumanEvalJava and LeetCodeJava). It is unclear how well CANDOR and the baselines (EvoSuite, LLM-Empirical, and TOGLL) generalize to other datasets, especially those that involve complex programs with dependencies on user-defined or external classes. However, HumanEvalJava is a representative, commonly used benchmark dataset in recent studies [14, 24, 44, 46, 48, 68, 74, 86], while we created LeetCodeJava to demonstrate the effectiveness of CANDOR on programs with higher and varying complexity. Moreover, our experimental results show that automated test generation, especially oracle generation, remains unresolved by existing approaches on these datasets. Our work thus represents an important first step towards generating tests for more complex programs. In fact, CANDOR is a multi-agent framework, which can be readily extended by introducing a dedicated agent to handle dependencies. Such an agent would retrieve definitions of user-defined or external classes, augmenting the test generation process when complex dependencies are involved. We leave this exploration for future work. Last, the strong performance of CANDOR on the LeetCodeJava dataset, which contains problems of varying difficulty, suggests that it is robust across diverse testing scenarios.

7 Related Work

We report prior work on test prefix generation and specification-based test oracle generation in Sections 7.1 and 7.2, respectively.

7.1 Test Prefix Generation

Early works in what is commonly referred to as test case generation are, in fact, more accurately characterized as test prefix generation accompanied by regression oracles. These works in test prefix generation primarily aim to automate manual testing by generating test prefixes with high code coverage. Traditional techniques in this area include fuzzing [26, 49], feedback-directed random test generation [7, 15, 52, 53, 60], dynamic symbolic execution [12, 30, 61, 75], and search/evolutionary algorithm-based approaches [27, 28, 52]. Among these approaches, EvoSuite has been extensively studied and applied to more than a hundred open-source software projects and several industrial systems, identifying thousands of potential bugs. However, as noted by Zhang et al. [87], traditional approaches such as EvoSuite often struggle to produce meaningful, human-readable test cases,

despite their remarkable code coverage. This is primarily due to their limited understanding of the semantics of the focal methods. To alleviate such issues, recent research has turned to LLMs to produce more practical test prefixes with high readability and coverage.

Tufano et al. [76] introduced the first LLM-based unit test generation approach AthenaTest, which formulates the task as a sequence-to-sequence learning problem. AthenaTest first denoises the pre-training of LLMs on a large Java corpus and then performs supervised fine-tuning for the downstream task of test generation. This work inspired subsequent research on fine-tuning LLMs for test generation [4, 36, 56]. Additionally, researchers have explored various strategies to improve LLM training for test case generation, including reinforcement learning [70], domain adaptation [65], and data augmentation [37].

Alternatively, a myriad of works perform prompt engineering using off-the-shelf LLMs, following a *generation-and-refinement* paradigm, where initial test cases are first generated based on prompts and then iteratively refined using dynamic execution feedback (e.g., code coverage report and failure information) [5, 14, 32, 50, 58, 59, 81, 86, 87]. To further improve the quality of test prefixes, researchers propose a wide range of strategies to incorporate LLMs with valuable contextual information, including mutation testing [18], method slicing [81], demonstration retrieval [90], defect detection [85], and program analysis [83].

Despite the rapid growth in LLM-based test generation research, Tang et al. [72] reported that EvoSuite remains the SOTA approach in generating test prefixes with high code coverage. CANDOR builds on prompt engineering-based test generation and makes the following contributions: (1) CANDOR is the first multi-agent LLM-based test generator. CANDOR lays the foundation for multi-agent frameworks in test generation by proposing a generic, effective framework with carefully designed agents, including *Planner*, *Tester*, and *Inspector*. (2) CANDOR presents a breakthrough in LLM-based test prefix generation, reaching comparable code coverage and significantly better mutation rates than evo.

7.2 Test Oracle Generation

Test oracle generation requires an accurate understanding of program semantics and requirements, posing significant challenges for early automated testing tools [5, 14, 28, 32, 50, 52, 58]. These tools circumvent these challenges by generating regression oracles, while specification-based oracle generation has been underexplored. Regression oracles are derived from the current implementation and are effective for detecting behavioral changes across software versions; however, they are limited in validating functional correctness against intended specifications. This limitation arises because regression oracles assume the existing code is correct, potentially masking bugs rather than detecting them.

However, going beyond regression oracles and generating specification-based oracles is challenging because it requires a deep understanding of functional requirements, edge cases, and expected outcomes [11]. With the advent of LLMs, researchers have begun to explore their potential for generating accurate specification-based oracles, using both prompt engineering and fine-tuning strategies. Dinella et al. [20] introduces TOGA, a transformer-based approach for inferring specification-based oracles from the context of the focal method. TOGA is the first LLM-based test oracle generator powered by a fine-tuned CodeBERT LLM. Inspired by this work, several studies have proposed generating specification-based oracles using either FT or PE strategies [24, 35, 35, 39, 56, 68]. Siddiq et al. [68] proposed LLM-Empirical, a solution using prompt engineering to generate JUnit tests based on natural language descriptions, without fine-tuning. It focuses on generating both test prefixes and oracles using only prompt engineering. Zhang et al. [89] conduct the first comprehensive study on fine-tuning various LLMs for oracle generation across two benchmarks, five LLMs, and two metrics. Building on this study, Zhang et al. [88] introduced RetriGen, a retrieval-augmented oracle generation approach that incorporates a novel hybrid assertion retriever into the oracle generation process. RetriGen retrieves most relevant assertions from external codebases by leveraging both lexical and semantic similarity. TOGLL[39] combines EvoSuite-generated test prefixes with

LLM-generated oracles through fine-tuned models, achieving SOTA performance in oracle generation for Java programs. However, TOGLL depends on test prefixes generated by EvoSuite and requires the extensive fine-tuning of large models, which can be resource-intensive and less adaptable to new domains.

CANDOR also relies on prompt engineering-based oracle generation but introduces several key innovations, including the design of multiple specialized agents, the panel discussion to reduce hallucination, and the dual-LLM pipeline to extract structured information from the verbose output of reasoning LLMs.

8 Conclusion

In this work, we presented CANDOR, a novel multi-agent, end-to-end framework for generating high-quality unit tests with accurate oracles. CANDOR does not require fine-tuning or the use of external tools but leverages a panel discussion to mitigate hallucinations and employs a dual-LLM pipeline to reduce overthinking during reasoning.

Extensive evaluation on two benchmarks, HumanEvalJava and LeetCode, demonstrates that CANDOR achieves performance comparable to EvoSuite in terms of line coverage and branch coverage, while substantially outperforming EvoSuite in mutation score. Additionally, CANDOR produces accurate specification-based oracles that outperform the state-of-the-art oracle generator TOGLL by a large margin (>= 21.1 percentage points). Ablation studies highlight the critical roles of key agents: the *Planner* for enhancing test prefix quality, and the *Requirement Engineer* together with the panel discussion for improving oracle correctness. For future work, we plan to extend CANDOR to project-level test generation and explore more advanced panel discussion strategies to further reduce hallucinations.

References

[1] Accessed: 2025. Langchain Official Website. https://www.langchain.com/

[2] Accessed: 2025. LeetCode Official Website. https://leetcode.com/

[3] Accessed: 2025. Pitest Official Website. https://pitest.org/

[4] Saranya Alagarsamy, Chakkrit Tantithamthavorn, and Aldeida Aleti. 2024. A3Test: Assertion-Augmented Automated Test case generation. *Information and Software Technology* 176 (2024), 107565. doi:10.1016/j.infsof.2024.107565

[5] Juan Altmayer Pizzorno and Emery D Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2897–2919.

[6] Andrea Arcuri and Lionel Briand. 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering. In *Proceedings of the 33rd international conference on software engineering*. 1–10.

[7] Ellen Arteca, Sebastian Harner, Michael Pradel, and Frank Tip. 2022. Nessie: Automatically testing javascript apis with asynchronous callbacks. In *Proceedings of the 44th International Conference on Software Engineering*. 1494–1505.

[8] Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868* (2022).

[9] AMOS O Bajeh, ONILEDE-JACOBS Oluwatosin, SHUIB Basri, ABIMBOLA G Akintola, and ABDULLATEEF O Balogun. 2020. Object-oriented measures as testability indicators: An empirical study. *J. Eng. Sci. Technol* 15, 2 (2020), 1092–1108.

[10] Kent Beck. 2022. *Test driven development: By example.* Addison-Wesley Professional.

[11] Soneya Binta Hossain, Raygan Taylor, and Matthew Dwyer. 2024. Doc2Oracle: Investigating the Impact of Javadoc Comments on Test Oracle Generation. *arXiv e-prints* (2024), arXiv–2412.

[12] Cristian Cadar, Vijay Ganesh, Peter M Pawlowski, David L Dill, and Dawson R Engler. 2008. EXE: Automatically generating inputs of death. *ACM Transactions on Information and System Security (TISSEC)* 12, 2 (2008), 1–38.

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[14] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576.

[15] Christoph Csallner and Yannis Smaragdakis. 2004. JCrasher: an automatic robustness tester for Java. *Software: Practice and Experience* 34, 11 (2004), 1025–1050.

[16] Ermira Daka, José Campos, Gordon Fraser, Jonathan Dorn, and Westley Weimer. 2015. Modeling readability to improve unit tests. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*. 107–118.

[17] Ermira Daka and Gordon Fraser. 2014. A survey on unit testing practices and problems. In *2014 IEEE 25th International Symposium on Software Reliability Engineering*. IEEE, 201–211.
[18] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468.
[19] Francisco Gomes de Oliveira Neto, Felix Dobslaw, and Robert Feldt. 2020. Using mutation testing to measure behavioural test diversity. In *2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 254–263.
[20] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K Lahiri. 2022. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering*. 2130–2141.
[21] Doocs. 2025. Coding Interview Solutions. https://github.com/doocs/leetcode. Accessed: 22 Nov, 2025.
[22] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 81, 13 pages. doi:10.1145/3597503.3639219
[23] Christof Ebert, James Cain, Giuliano Antoniol, Steve Counsell, and Phillip Laplante. 2016. Cyclomatic complexity. *IEEE software* 33, 6 (2016), 27–29.
[24] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K Lahiri. 2024. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1889–1912.
[25] Andrew Ettles, Andrew Luxton-Reilly, and Paul Denny. 2018. Common logic errors made by novice programmers. In *Proceedings of the 20th Australasian Computing Education Conference*. 83–89.
[26] Andrea Fioraldi, Alessandro Mantovani, Dominik Maier, and Davide Balzarotti. 2023. Dissecting american fuzzy lop: a fuzzbench evaluation. *ACM transactions on software engineering and methodology* 32, 2 (2023), 1–26.
[27] Gordon Fraser and Andrea Arcuri. 2011. Evolutionary generation of whole test suites. In *2011 11th International Conference on Quality Software*. IEEE, 31–40.
[28] Gordon Fraser and Andrea Arcuri. 2011. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*. 416–419.
[29] Gordon Fraser and Andrea Arcuri. 2014. A Large-Scale Evaluation of Automated Unit Test Generation Using EvoSuite. *ACM Trans. Softw. Eng. Methodol.* 24, 2, Article 8 (Dec. 2014), 42 pages. doi:10.1145/2685612
[30] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation*. 213–223.
[31] Danielle Gonzalez, Joanna CS Santos, Andrew Popovich, Mehdi Mirakhorli, and Mei Nagappan. 2017. A large-scale study on the usage of testing patterns that address maintainability attributes: patterns for ease of modification, diagnoses, and comprehension. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 391–401.
[32] Siqi Gu, Chunrong Fang, Quanjun Zhang, Fangyuan Tian, and Zhenyu Chen. 2024. Testart: Improving llm-based unit test via co-evolution of automated generation and repair iteration. *arXiv e-prints* (2024), arXiv–2408.
[33] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
[34] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680* (2024).
[35] Ishrak Hayet, Adam Scott, and Marcelo d'Amorim. 2025. ChatAssert: LLM-Based Test Oracle Generation With External Tools Assistance. *IEEE Transactions on Software Engineering* 51, 1 (2025), 305–319. doi:10.1109/TSE.2024.3519159
[36] Yifeng He, Jiabo Huang, Yuyang Rong, Yiwen Guo, Ethan Wang, and Hao Chen. 2024. UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1061–1072. doi:10.1145/3650212.3680342
[37] Yifeng He, Jicheng Wang, Yuyang Rong, and Hao Chen. 2024. Data Augmentation by Fuzzing for Neural Test Generation. *arXiv preprint arXiv:2406.08665* (2024).
[38] Hadi Hemmati. 2015. How effective are code coverage criteria?. In *2015 IEEE International Conference on Software Quality, Reliability and Security*. IEEE, 151–156.
[39] Soneya Binta Hossain and Matthew Dwyer. 2024. Togll: Correct and strong test oracle generation with llms. *arXiv preprint arXiv:2405.03786* (2024).
[40] Min Hua, Qiyu Lu, Bin Hua, Ping Song, and Ling Luo. 2025. Research on Fine-Tuning and Prompt Engineering of LLMs for Scientific Literature in Power Grid. In *2025 5th International Conference on Artificial Intelligence, Big Data and Algorithms (CAIBDA)*. IEEE, 1587–1591.
[41] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM*

Transactions on Information Systems 43, 2 (2025), 1–55.

[42] Charles Ikerionwu. 2010. CYCLOMATIC COMPLEXITY AS A SOFTWARE METRIC. *International Journal of Academic Research* 2, 3 (2010).

[43] Kush Jain and Claire Le Goues. 2025. TestForge: Feedback-Driven, Agentic Test Suite Generation. *arXiv preprint arXiv:2503.14713* (2025).

[44] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1430–1442.

[45] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis* (San Jose, CA, USA) *(ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055

[46] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. 2025. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering* 30, 2 (2025), 45.

[47] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931.

[48] Fengjie Li, Jiajun Jiang, Jiajun Sun, and Hongyu Zhang. 2024. Hybrid automated program repair by combining large language models and program analysis. *ACM Transactions on Software Engineering and Methodology* (2024).

[49] Barton P Miller, Lars Fredriksen, and Bryan So. 1990. An empirical study of the reliability of UNIX utilities. *Commun. ACM* 33, 12 (1990), 32–44.

[50] Chao Ni, Xiaoya Wang, Liushan Chen, Dehai Zhao, Zhengong Cai, Shaohua Wang, and Xiaohu Yang. 2024. CasModaTest: A cascaded and model-agnostic self-directed framework for unit test generation. *arXiv preprint arXiv:2406.15743* (2024).

[51] TIOBE Organization. 2025. Which programming language produces the most complex code? https://www.tiobe.com/knowledge/article/which-programming-language-produces-the-most-complex-code/. Accessed: 22 Nov, 2025.

[52] Carlos Pacheco and Michael D Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion*. 815–816.

[53] Carlos Pacheco, Shuvendu K Lahiri, and Thomas Ball. 2008. Finding errors in. net with feedback-directed random testing. In *Proceedings of the 2008 international symposium on Software testing and analysis*. 87–96.

[54] Carlos Pacheco, Shuvendu K Lahiri, Michael D Ernst, and Thomas Ball. 2007. Feedback-directed random test generation. In *29th International Conference on Software Engineering (ICSE'07)*. IEEE, 75–84.

[55] Rishov Paul, Md Mohib Hossain, Mohammed Latif Siddiq, Masum Hasan, Anindya Iqbal, and Joanna Santos. 2023. Enhancing automated program repair through fine-tuning and prompt engineering. *arXiv preprint arXiv:2304.07840* (2023).

[56] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM Training Language Models on Aligned Code And Tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 409–420. doi:10.1109/ASE56229.2023.00193

[57] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting using LLM. *Proc. ACM Softw. Eng.* 1, FSE, Article 43 (July 2024), 21 pages. doi:10.1145/3643769

[58] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971.

[59] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105.

[60] Marija Selakovic, Michael Pradel, Rezwana Karim, and Frank Tip. 2018. Test generation for higher-order functions in dynamic languages. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–27.

[61] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A concolic unit testing engine for C. *ACM SIGSOFT software engineering notes* 30, 5 (2005), 263–272.

[62] Domenico Serra, Giovanni Grano, Fabio Palomba, Filomena Ferrucci, Harald C Gall, and Alberto Bacchelli. 2019. On the effectiveness of manual and automatic unit test generation: ten years later. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 121–125.

[63] Sina Shamshiri, René Just, José Miguel Rojas, Gordon Fraser, Phil McMinn, and Andrea Arcuri. 2015. Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 201–211.

[64] August Shi, Jonathan Bell, and Darko Marinov. 2019. Mitigating the effects of flaky tests on mutation testing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 112–122.

[65] Jiho Shin, Sepehr Hashtroudi, Hadi Hemmati, and Song Wang. 2024. Domain Adaptation for Code Model-Based Unit Test Case Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA*

2024). Association for Computing Machinery, New York, NY, USA, 1211–1222. doi:10.1145/3650212.3680354

[66] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2023. Prompt engineering or fine tuning: An empirical assessment of large language models in automated software engineering tasks. *arXiv preprint arXiv:2310.10508* (2023).

[67] Jiho Shin, Clark Tang, Tahmineh Mohati, Maleknaz Nayebi, Song Wang, and Hadi Hemmati. 2025. Prompt Engineering or Fine-Tuning: An Empirical Assessment of LLMs for Code. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 490–502.

[68] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. 2024. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 313–322.

[69] Maxim Stavtsev and Yegor Bugayenko. 2024. Evaluating the Dependency Between Cyclomatic Complexity and Response For Class. *arXiv preprint arXiv:2410.06416* (2024).

[70] Benjamin Steenhoek, Michele Tufano, Neel Sundaresan, and Alexey Svyatkovskiy. 2025. Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation . In *2025 IEEE/ACM International Workshop on Deep Learning for Testing and Testing for Deep Learning (DeepTest)*. IEEE Computer Society, Los Alamitos, CA, USA, 37–44. doi:10.1109/DeepTest66595.2025.00011

[71] Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, et al. 2025. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv preprint arXiv:2503.16419* (2025).

[72] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. Chatgpt vs sbst: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1340–1359.

[73] NN Thathsarani. 2024. A comprehensive software complexity metric based on cyclomatic complexity. In *2024 4th International Conference of Science and Information Technology in Smart Administration (ICSINTESA)*. IEEE, 90–95.

[74] Zhao Tian, Junjie Chen, and Xiangyu Zhang. 2025. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 645–645.

[75] Nikolai Tillmann, Jonathan De Halleux, and Tao Xie. 2014. Transferring an automated test generation tool to practice: From Pex to Fakes and Code Digger. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*. 385–396.

[76] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, Shao Kun Deng, and Neel Sundaresan. 2020. Unit test case generation with transformers and focal context. *arXiv preprint arXiv:2009.05617* (2020).

[77] Jonathan G Tullis and Aaron S Benjamin. 2011. On the effectiveness of self-paced learning. *Journal of memory and language* 64, 2 (2011), 109–118.

[78] Guancheng Wang, Qinghua Xu, Lionel C Briand, and Kui Liu. 2025. Mutation-Guided Unit Test Generation with a Large Language Model. *arXiv preprint arXiv:2506.02954* (2025).

[79] Han Wang, Han Hu, Chunyang Chen, and Burak Turhan. 2024. Chat-like asserts prediction with the support of large language model. *arXiv preprint arXiv:2407.21429* (2024).

[80] Xinyi Wang, Qinghua Xu, Paolo Arcaini, Shaukat Ali, and Thomas Peyrucain. 2025. Quantum machine learning-based test oracle for autonomous mobile robots. *arXiv preprint arXiv:2508.02407* (2025).

[81] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. Hits: High-coverage llm-based unit test generation via method slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268.

[82] Jiahui Wu. 2025. Uncertainty-Aware Autonomous Driving System Testing with Large Language Models. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 776–778.

[83] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing llm-based test generation for hard-to-cover branches via program analysis. *arXiv preprint arXiv:2404.04966* (2024).

[84] Lin Yang, Chen Yang, Shutao Gao, Weijing Wang, Bo Wang, Qihao Zhu, Xiao Chu, Jianyi Zhou, Guangtai Liang, Qianxiang Wang, et al. 2024. On the evaluation of large language models in unit test generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1607–1619.

[85] Xin Yin, Chao Ni, Xiaodan Xu, and Xiaohu Yang. 2025. What You See is What You Get: Attention-Based Self-Guided Automatic Unit Test Generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1039–1051.

[86] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE, Article 76 (July 2024), 24 pages. doi:10.1145/3660783

[87] Quanjun Zhang, Chunrong Fang, Siqi Gu, Ye Shang, Zhenyu Chen, and Liang Xiao. 2025. Large Language Models for Unit Testing: A Systematic Literature Review. *arXiv preprint arXiv:2506.15227* (2025).

[88] Quanjun Zhang, Chunrong Fang, Yi Zheng, Yaxin Zhang, Yuan Zhao, Rubing Huang, Jianyi Zhou, Yun Yang, Tao Zheng, and Zhenyu Chen. 2025. Improving Deep Assertion Generation via Fine-Tuning Retrieval-Augmented Pre-trained Language Models. *ACM Transactions on Software Engineering and Methodology* (2025).

[89] Quanjun Zhang, Weifeng Sun, Chunrong Fang, Bowen Yu, Hongyan Li, Meng Yan, Jianyi Zhou, and Zhenyu Chen. 2025. Exploring automated assertion generation via large language models. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–25.

[90] Zhe Zhang, Xingyu Liu, Yuanzhang Lin, Xiang Gao, Hailong Sun, and Yuan Yuan. 2024. LLM-based Unit Test Generation via Property Retrieval. *arXiv preprint arXiv:2410.13542* (2024).

Appendix

CANDOR with Alternative LLMs

Dataset	Metric	Llama	CodeLlama	Mistral
HumanEvalJava	Line	**0.991**	0.991	0.840*
	Branch	**0.970**	0.967	0.839*
	Mutation	**0.980** (2384/2443)	0.980 (2390/2443)	0.796* (1945/2443)
	Oracle	0.910	**0.912**	0.861*
Leetcode-Medium	Line	**0.990**	0.990	0.914*
	Branch	0.949	**0.950**	0.943*
	Mutation	0.939 (574/611)	**0.948** (579/611)	0.839* (513/611)
	Oracle	**0.930**	0.919	0.904*
Leetcode-Hard	Line	0.989	**0.990**	0.876*
	Branch	0.980	**0.981**	0.871*
	Mutation	**0.937** (805/859)	0.930 (799/859)	0.905* (777/859)
	Oracle	**0.894**	0.869*	0.848*

Table 3. Experimental results of CANDOR using alternative LLMs. "Llama", "CodeLlama", and "Mistral" denote CANDOR using *Llama 70B*, *CodeLlama 70B*, and *Mistral 22b*, respectively. The highest value in each row is highlighted in bold. The "*" indicates that the difference between this result and CANDOR using Llama 70B is statistically significant.

Table 3 reports the experimental results of CANDOR using alternative LLMs. We observe that Mistral 22B has the lowest effectiveness among the three models, resulting in substantially lower line coverage, branch coverage, mutation scores, and oracle correctness. Wilcoxon Signed Rank tests confirm that all differences between Mistral 22B and Llama 70B are statistically significant, with p-values below 10^{-4}. This outcome is expected, as Mistral 22B has considerably fewer parameters than Llama 70B and CodeLlama 70B, making it less capable across tasks, including test generation.

In contrast, CodeLlama 70B and Llama 70B yield comparable results. For test prefix generation, the largest difference between the two models is a 0.09 gap in mutation score on the LeetCode-Medium dataset. However, Wilcoxon Signed Rank tests show that none of these differences are statistically significant. For oracle generation, Llama 70B achieves slightly higher correctness on LeetCode-Medium ($0.011 = 0.930 - 0.919$) and LeetCode-Hard ($0.025 = 0.894 - 0.869$), while performing slightly worse on HumanEvalJava ($0.002 = 0.912 - 0.910$). The differences between HumanEvalJava and LeetCode-Medium are not statistically significant, whereas the difference between LeetCode-Hard and HumanEvalJava is. Overall, Llama 70B performs marginally better than CodeLlama 70B. For this reason, we select Llama 70B as the default model for CANDOR and report the corresponding results in the main manuscript.

Received 29 August 2025; revised 8 December 2025; accepted 13 March 2026